# CS336 Assignment 3: Writeup

Jack Le

Spring 2026

## 2.1 `chinchilla_isoflops`

(a) Show your extrapolated compute-optimal model size, together with the $\langle C_i, N_{\text{opt}}(C_i)\rangle$ points you obtained. What is your predicted optimal model size for a budget of $10^{23}$ FLOPs? What about for $10^{24}$ FLOPs?

**Answer:** The red points are the observed $\langle C_i, N_{\text{opt}}(C_i)\rangle$ points, and the blue line is the fitted scaling law. With a budget of $10^{23}$ FLOPs, the predicted optimal model size is $5.00 \times 10^{10}$ parameters. With $10^{24}$ FLOPs, the predicted optimal model size is $1.27 \times 10^{11}$ parameters.

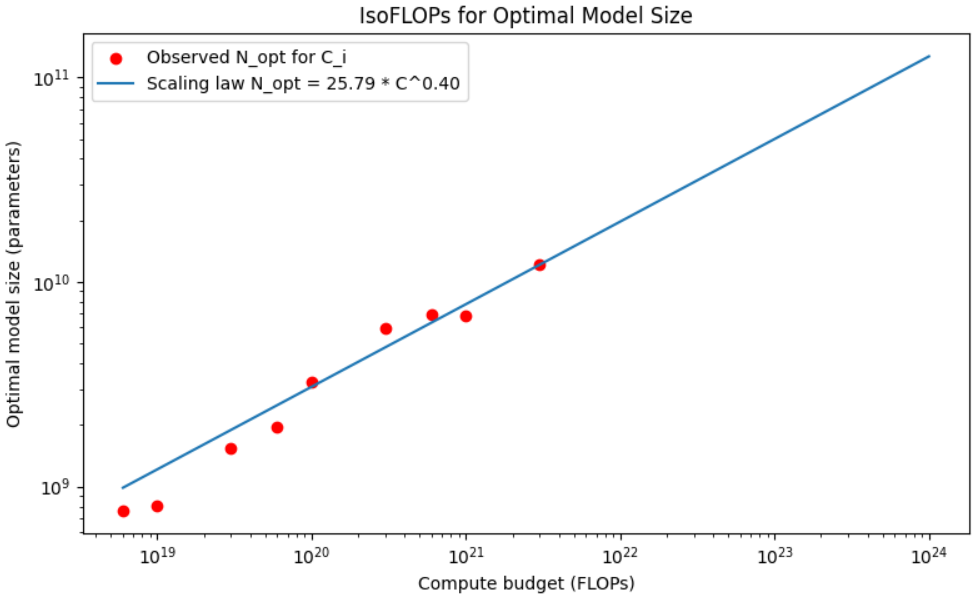


Figure 1: Extrapolated compute-optimal model size

(b) Show your extrapolated compute-optimal dataset size, together with the $\langle C_i, D_{\text{opt}}(C_i)\rangle$ data points from the training runs. What is your predicted optimal dataset size for budgets of $10^{23}$ and $10^{24}$ FLOPs?

**Answer:** The blue points are the observed $\langle C_i, D_{\text{opt}}(C_i)\rangle$ points, and the black line is the fitted scaling law. With a budget of $10^{23}$ FLOPs, the predicted optimal dataset size is $3.37 \times 10^{11}$ tokens. With $10^{24}$ FLOPs, the predicted optimal dataset size is $1.33 \times 10^{12}$ tokens.

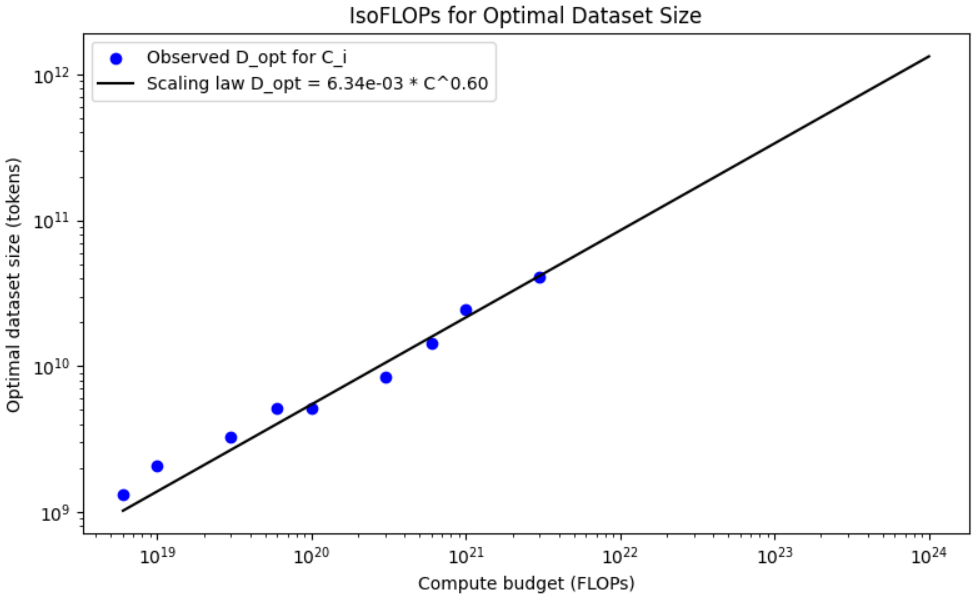


Figure 2: Extrapolated compute-optimal dataset size

## 3 `scaling_laws`

**Answer:** In my final submission, I approached this in three stages: (1) fit a learning rate scaling law, (2) run IsoFLOP experiments to fit scaling laws for optimal model size and loss, and (3) extrapolate to the full 48-hour budget. However, my first attempt wasted the majority of my budget, forcing me to restart with a significantly reduced compute allowance. The code for the final submission is at `attempt2.ipynb`. The code for the first attempt is at `scaling_laws.ipynb`.

### First Attempt

In my initial approach, I naively ran an IsoFLOP grid across compute budgets of $10^{16}$, $3 \times 10^{16}$, $10^{17}$, and $3 \times 10^{17}$ FLOPs using a fixed learning rate of $3 \times 10^{-4}$ for all model sizes. This was the default from the example configuration. The resulting IsoFLOP curves were essentially flat and no U-shape was visible (Figure 3). After looking into it further, I realized that using a fixed learning rate is far from optimal since smaller models prefer higher learning rates and larger models prefer lower learning rates. When I reran some configurations at $10^{-3}$, the loss improved dramatically, confirming that the LR was the dominant factor rather than the model size/data trade-off.

This consumed approximately 10 hours of my 12-hour budget, leaving me with roughly 2 hours. After discussing with Marcel, I decided to restart and plan it out much more carefully. Namely, I wanted to

first fit the LR scaling law, then use it to set per-model learning rates for the IsoFLOP experiments.

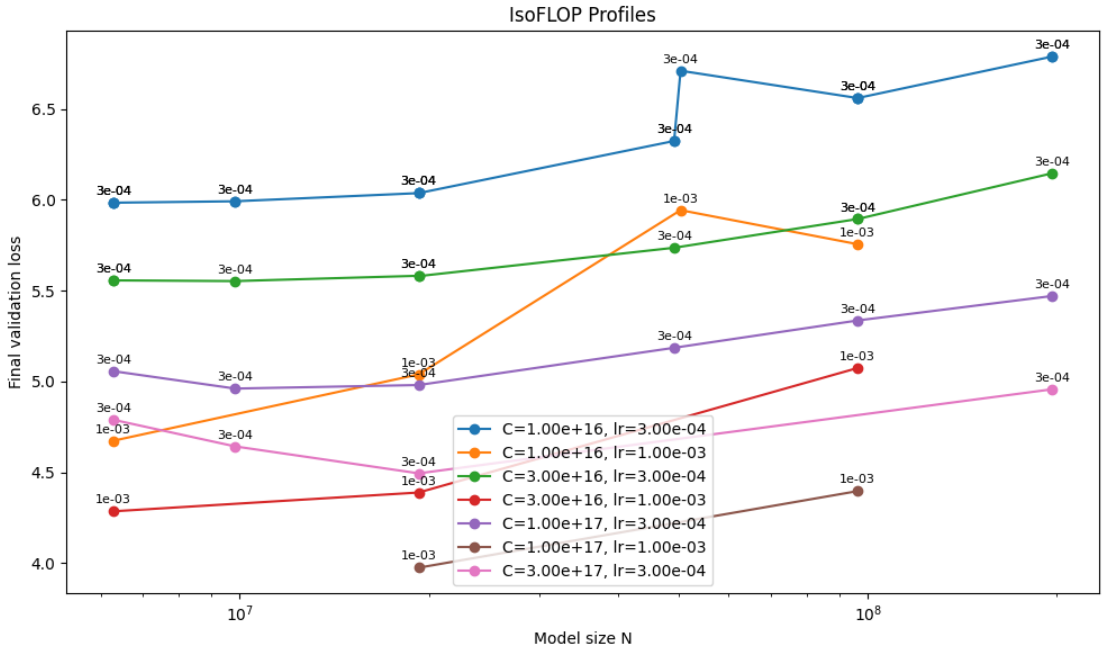


Figure 3: IsoFLOP curves from the first attempt with fixed LR $= 3 \times 10^{-4}$

## Learning Rate Scaling Law

With my remaining budget, I first swept 9 learning rates roughly spaced out from $10^{-4}$ to $3 \times 10^{-2}$ across 4 model sizes (1M, 5M, 15M, and 50M non-embedding parameters), each trained for 300 steps. Figure 4 shows the loss vs. LR for each model size, matching the expected trend of lower learning rates for larger models. We also see that my initial LR of $3 \times 10^{-4}$ was suboptimal for all models.

For each model size, I selected the LR with the lowest loss, then fit a power law:

$$\text{lr}_{\text{opt}} = 2.1166 \cdot N^{-0.3893}$$

This was used to set the learning rate for all subsequent experiments.

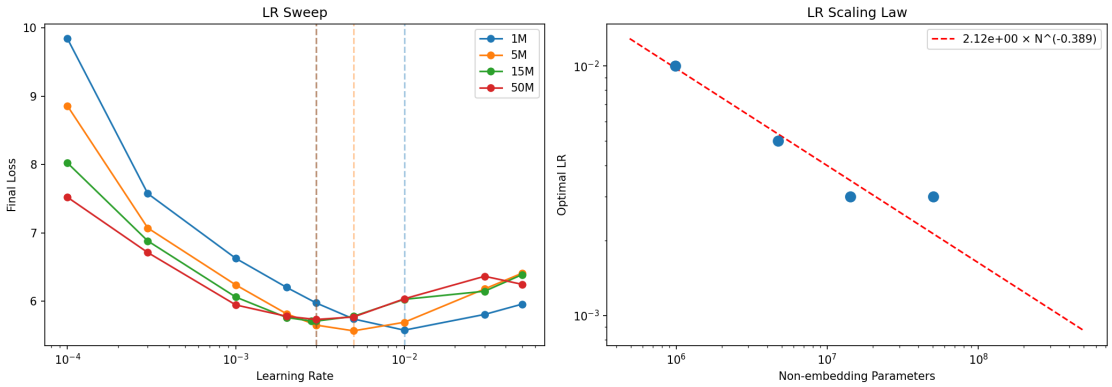


Figure 4: Left: validation loss vs. learning rate for each model size. Right: optimal learning rate vs. model size with fitted power law.

## IsoFLOP Experiments

Given my reduced budget, I lowered the compute budgets considerably compared to my first attempt. I ran IsoFLOP experiments across budgets ranging from $10^{14}$ to $3 \times 10^{16}$ FLOPs. At each budget $C$, I trained models of varying size $N$, with tokens set as $D = C/(6N)$ rounded to a valid batch multiple. The learning rate for each model was set using the scaling law.

I used model configurations spanning 0.2M to 200M non-embedding parameters, varying `hidden_size` (64 to 1280) and `num_hidden_layers` (2 to 14). I calculated the intermediate size as $\lceil 8d/3 \rceil$ rounded up to a multiple of 64 based on the SwiGLU from assignment 1. All other hyperparameters were held fixed: batch size 128, weight decay 0.01, $\beta_1 = 0.9$, $\beta_2 = 0.95$, warmup fraction 0.05, final LR fraction 0.1, and bfloat16 precision.

I set strict time limits on each run. This caused several experiments to time out, particularly those with small model sizes and correspondingly large token counts. These failed runs are marked with stars in Figure 5. Despite these gaps, the IsoFLOP curves show clear U-shapes at each compute budget.

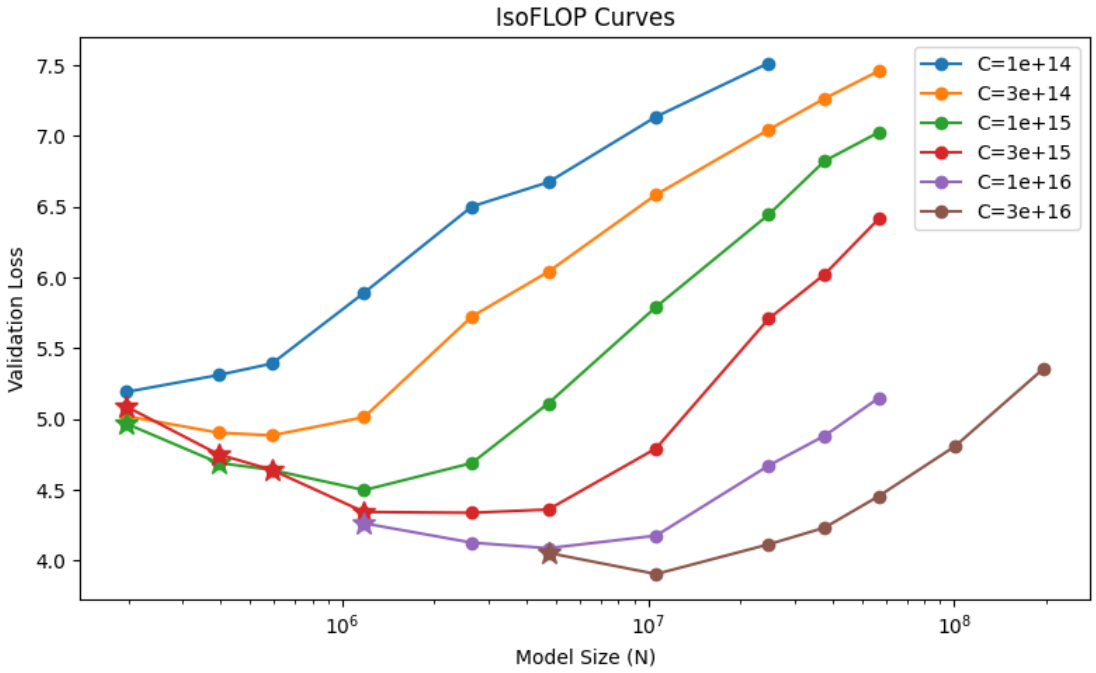


Figure 5: IsoFLOP curves from the second attempt. Stars indicate runs that timed out.

## Fitting Scaling Laws

Using the optimal model size and loss from the IsoFLOP experiments, I fit the following scaling laws.

$$N_{\text{opt}} = 1.52 \times 10^{-4} \cdot C^{0.658}$$

$$D_{\text{opt}} = 7.38 \times 10^{2} \cdot C^{0.353}$$

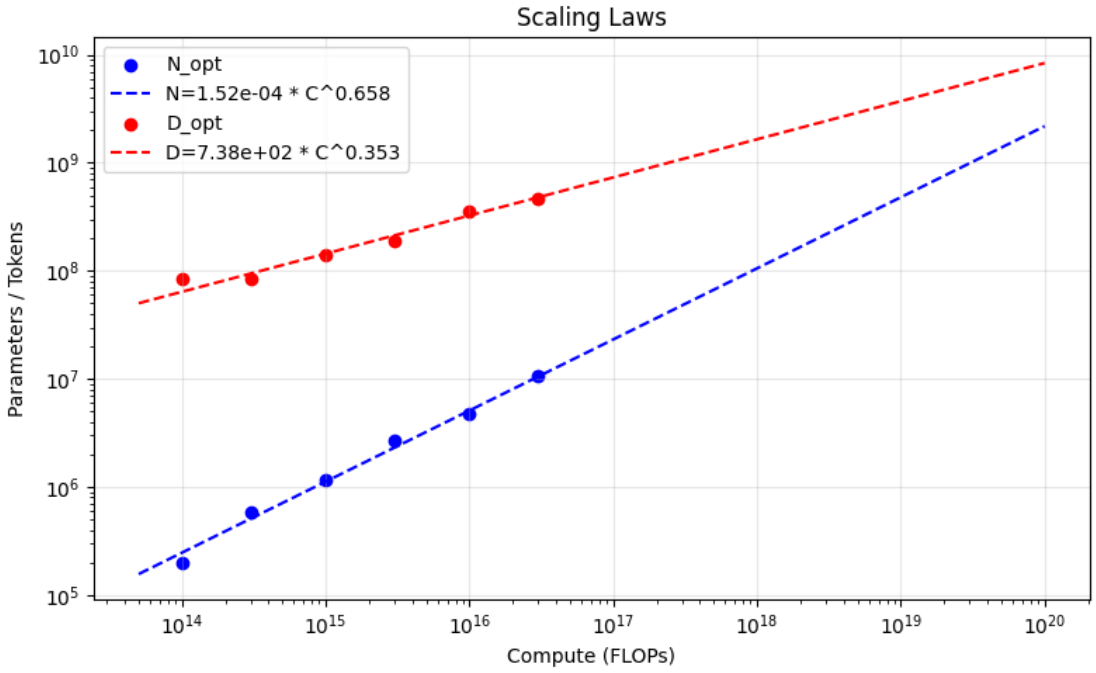


Figure 6: Optimal model size and dataset size vs. compute budget with fitted power laws

## Estimating Compute Budget

To convert 48 B200-hours into FLOPs, I estimated GPU throughput from the runtimes of completed experiments. Across both attempts, I had collected runtimes from 120 runs. Of these, 37 were with $N > 50$M parameters where GPU utilization is more representative of the final model. I computed an average throughput of approximately $2.06 \times 10^{14}$ FLOPS, giving an estimate of:

$$C_{48\text{h}} = 2.06 \times 10^{14} \times 48 \times 3600 = 3.56 \times 10^{19} \text{ FLOPs}$$

The average thruoghput was calculated by computing $6 \times N \times D$ for each run and dividing by the runtime.

## Loss Scaling Law

With the compute budgets estimated, I used the IsoFLOP results to fit the loss scaling law. I identified the lowest observed validation loss for each budget and then fit these to a power law, which gave me:

$$L_{\text{opt}} = 2.5755 \times 10^2 \cdot C^{-0.1469} + 2.9332$$

where the constant term $2.93$ represents the irreducible loss. This gave a predicted loss of **3.2795** at the estimated budget of $3.56 \times 10^{19}$ FLOPs.

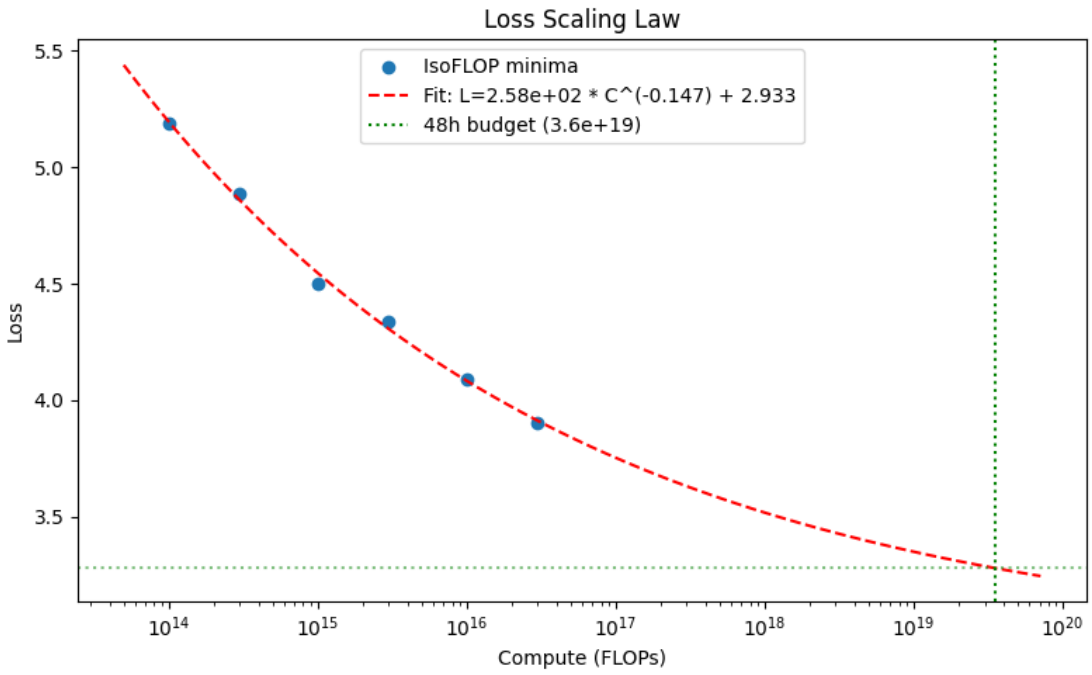


Figure 7: Optimal loss vs. compute budget with fitted scaling law

## Final Prediction

Applying the scaling laws to the estimated budget gave me the following predictions:

| Quantity | Predicted Value |
|---|---|
| Compute budget | $3.56 \times 10^{19}$ FLOPs |
| Optimal model size ($N$) | $1.11 \times 10^{9}$ parameters |
| Optimal tokens ($D$) | $5.82 \times 10^{9}$ tokens |
| Learning rate | $6.38 \times 10^{-4}$ |
| Predicted loss | 3.2753 |

This prediction is slightly over the budget at $3.86 \times 10^{19}$ FLOPs. However, given that it's better in this case to run through the budget than to stop early, I decided to use this prediction.

I selected an architecture with `hidden_size` $= 2048$, `num_hidden_layers` $= 22$, giving $N = 12 \times 22 \times 2048^2 \approx 1.11$B non-embedding parameters. The alternatives that I considered were $d = 1536, L = 39$ and $d = 1024, L = 88$, but I ended up choosing this one because it's similar to the LLaMA 3 architecture which was been shown to work well.

Given more budget, I would've liked to experiment further with tied embeddings and batch size since they've been shown to be important for performance.